# Hadoop & Spark - ISEN 2025/2026

## Chapter 2: Hadoop

# Goals of this chapter:

In this chapter we will see:

- The need for separation between compute and storage
- Learn about HDFS and Hadoop

# Lets start simple: store a file locally

One User, one computer, one disk: easy!

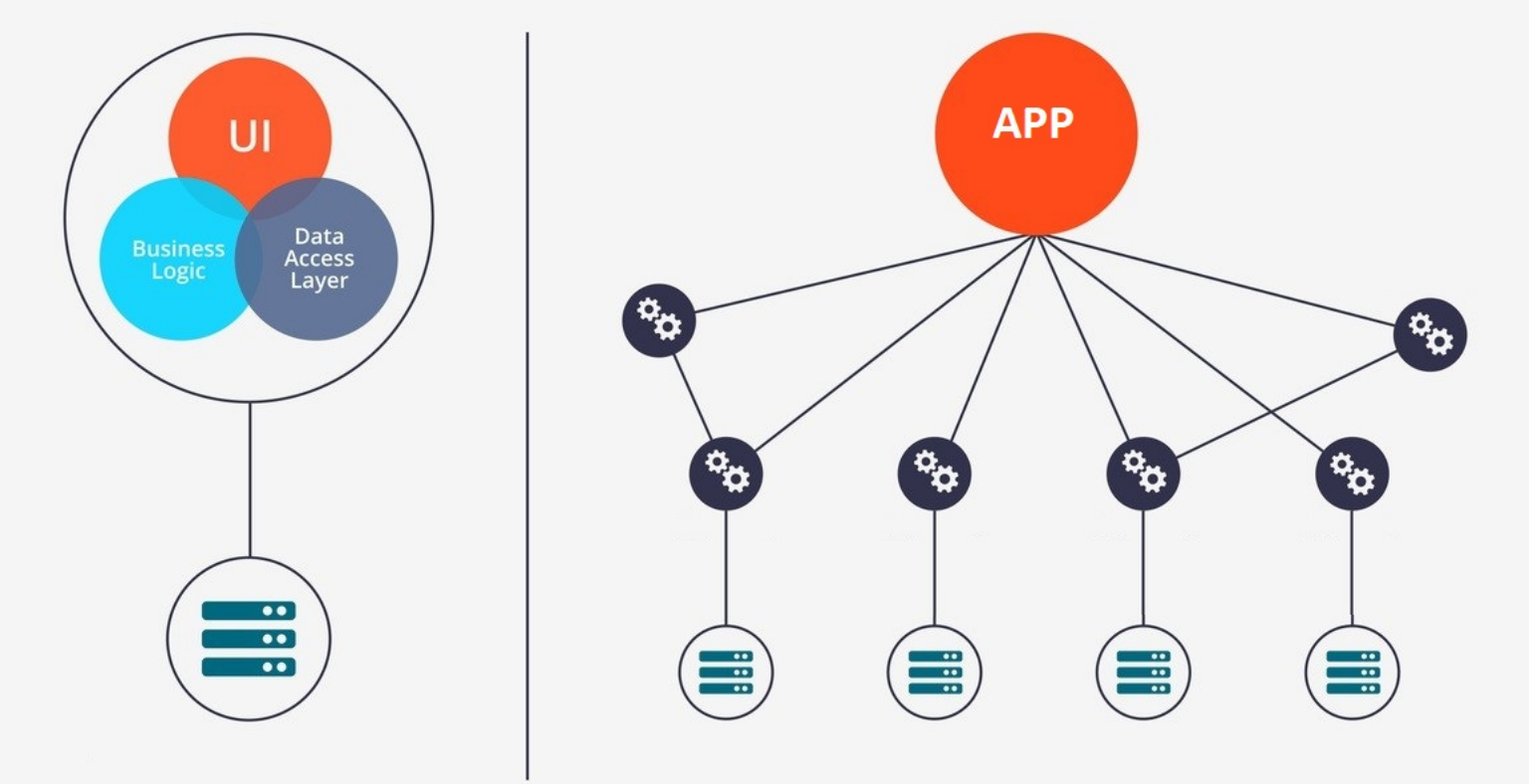
But this works only locally when you have one user, one computer, and one drive: In a big data scenario, you will have up to millions of users, and this raises some challenges



# A need for separation between computing and storage

When you build an application (including a Data application) there are two main opposing architectures:

**Monolithic** & **Distributed**

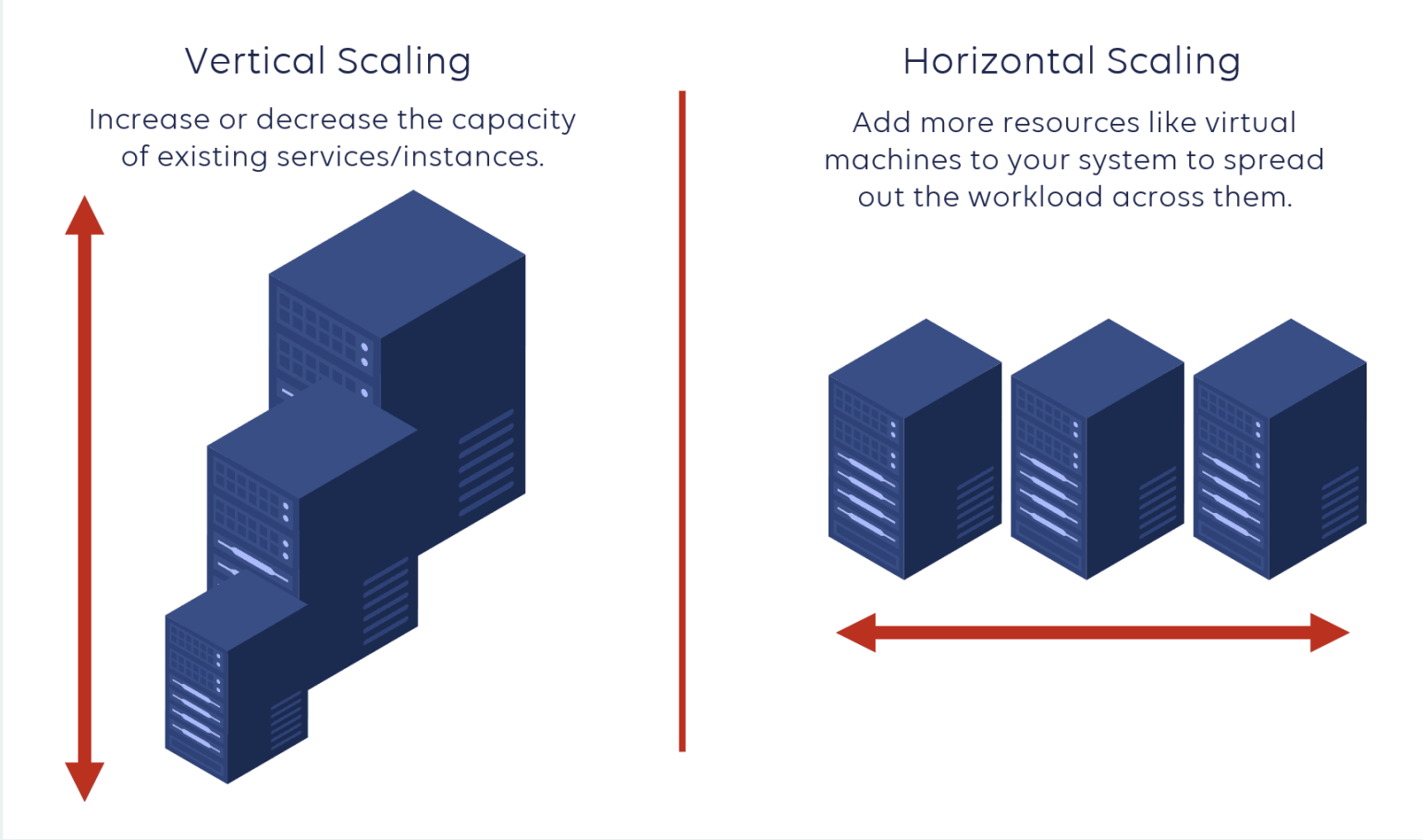


# Monolithic VS Distributed: Scalability

- **Monolithic system**:
  - Compute and storage resources are combined in a single unit, making it difficult to scale one without the other.
- **Distributed system**:
  - Allow to scale compute and storage resources independently.
  - This separation enables you to add more compute nodes or storage capacity as needed without affecting the other resources.

# Vertical & Horizontal scaling

Vertical scaling refers to increasing a system's capacity by adding more resources, such as CPU or memory, to a single node, while horizontal scaling involves adding more nodes to a system to distribute the workload across multiple machines.



# Monolithic VS Distributed: Performance

- **Monolithic system**:
  - The processing power of the system is often limited by the storage capacity.
  - As data grows, the performance of the system may degrade due to bottlenecks in storage.
- **Distributed system**:
  - By decoupling computing and storage in a distributed architecture, each component can be optimized for its specific purpose, improving overall performance.
  - For example, compute nodes can be provisioned with more memory and processing power, while storage nodes can be optimized for capacity and I/O throughput.

# Monolithic VS Distributed: Flexibility

- **Monolithic system**:
  - You need to choose one single technology for the whole application.
- **Distributed system**:
  - Decoupling computing and storage allows you to choose the right technology and infrastructure for each component.
  - For example, you can use different types of storage systems (e.g., object storage, file systems, or block storage) depending on your data access patterns.

# Monolithic VS Distributed: Fault Tolerance

- **Monolithic system**:
  - A failure in the storage or compute component can impact the entire system, causing downtime and data loss.
- **Distributed system**:
  - The system is designed to handle failures in individual components without affecting the overall system.
  - Data is often replicated across multiple storage nodes, providing redundancy and fault tolerance.
  - Computes nodes may also be configured for high availability.

# Monolithic VS Distributed: Cost Efficiency

- **Monolithic system**:
  - One fixed size
- **Distributed system**:
  - You can optimize resource provisioning based on your workload's actual needs, which can lead to cost savings.

# So why do we even bother with monolithic if distributed is that awesome?



# Monolithic VS Distributed: Complexity

- **Monolithic system**:
  - One single system to maintain and manage. Easy.
- **Distributed system**:
  - Maintaining data consistency across multiple nodes can be challenging.
  - Coordinating tasks and managing resources among multiple nodes requires additional complexity.
  - Deploying, configuring, and managing can be complex due to different configurations.
  - Troubleshooting issues requires you to analyze logs and metrics from multiple nodes at the same time.

# Complexity: introducing clustering and computing frameworks

With an adapted distributed processing framework, the application (and the developer) does not need to deal with the distributed aspect. Everything is handled by the software, given we have configured it properly.



# One example of software implementation: introducing Hadoop

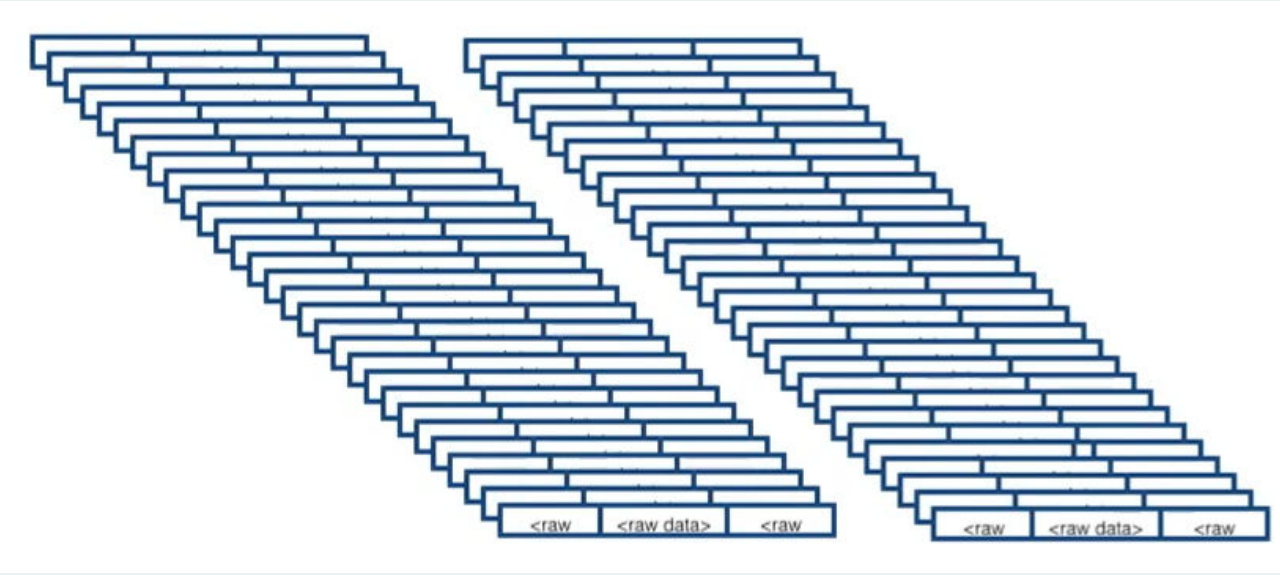
In the early 2000s, Google was trying to create its search engine and realized that it would require a completely new kind of system that did not exist at the time for such a scale of data.

They needed something distributed to scale and cover the needs of users all over the world.

**There were two main objectives.**

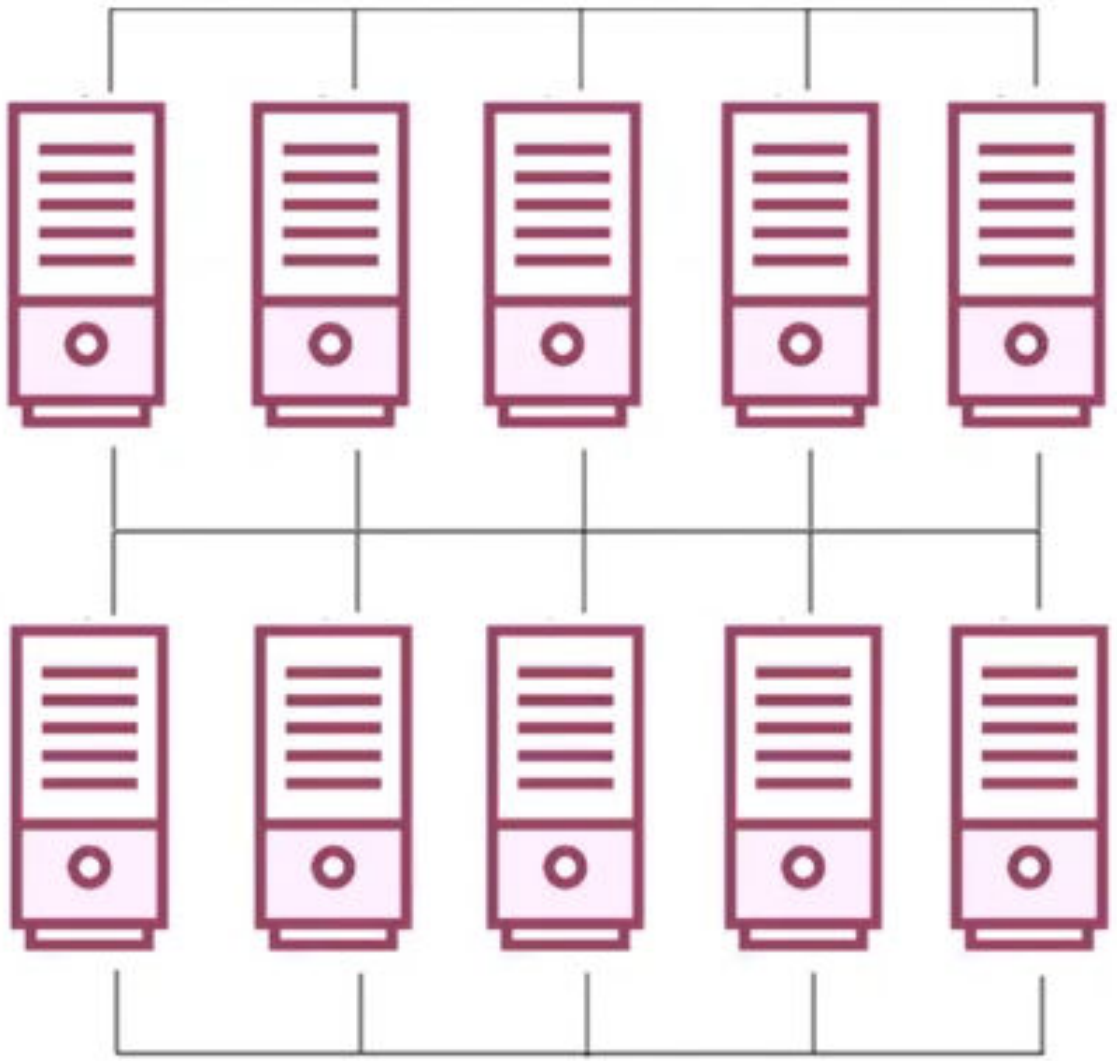
# Objective n°1

Store millions of records on multiple machines and several locations



# Objective n°2

A process would need to be coordinated across multiple machines in a distributed manner to accomplish a single task



# The answer from Google to those 2 objectives:

- Google File System -> To solve distributed storage
- MapReduce -> To solve distributed computing



# Apache made an open-sourced version: Hadoop

| Google | Apache (open source) |
| --- | --- |
| Google File System | Hadoop Distributed File System (HDFS) |
| MapReduce | MapReduce (Stunning right ? 🤯) |




This gave birth to the Hadoop framework.

# Apache made an open-sourced version: Hadoop

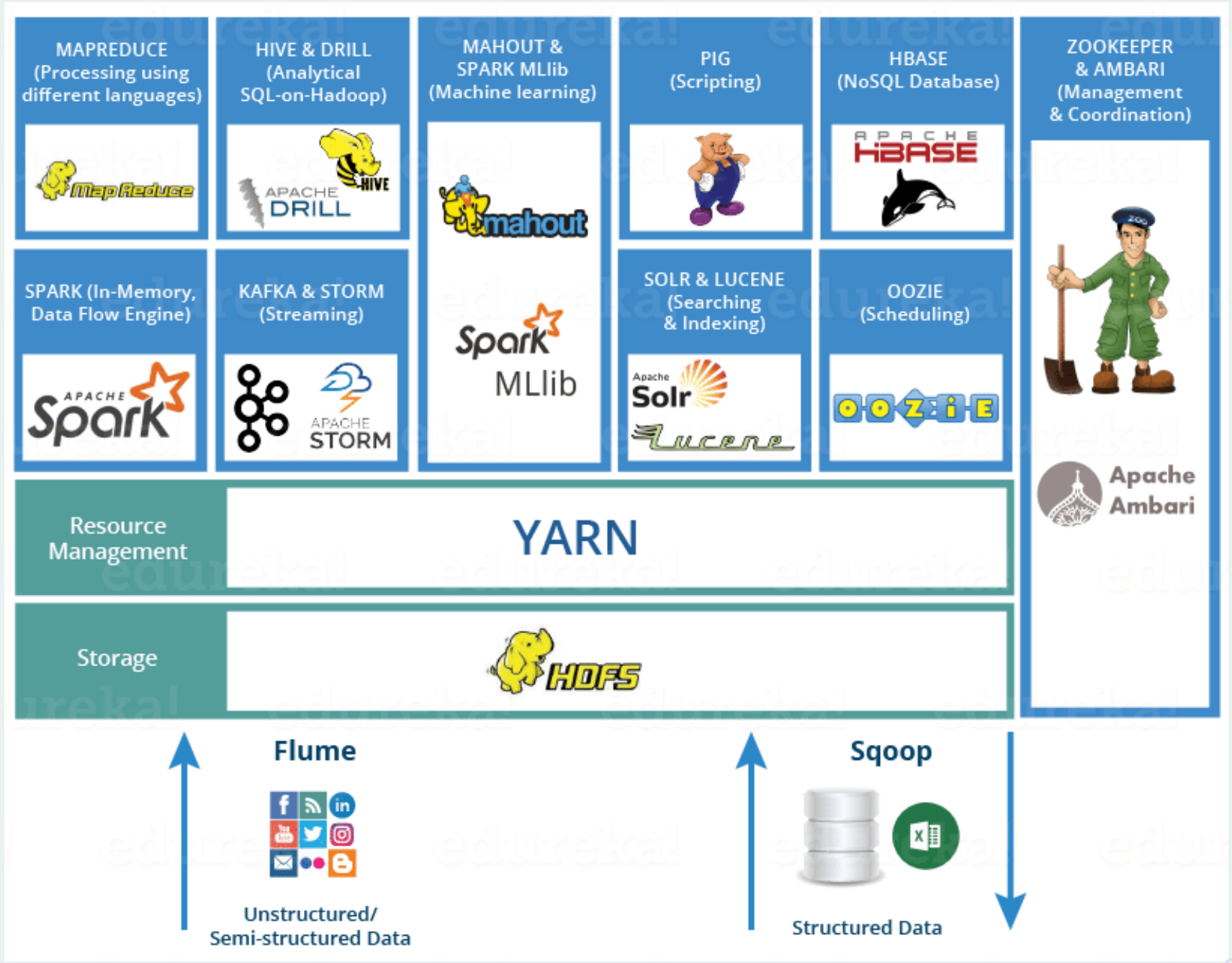
- Apache later decided (2013) to split their MapReduce implementation into two additionnal blocks, giving us those 3 layers:
  - HDFS -> Storage layer
  - MapReduce -> Processing layer (Plan the execution)
  - YARN -> Resource Management layer (Execute the plan on the machines)

  All of this happened to workers and nodes.



# Hadoop ecosystem overview

Those are only examples, other tools and frameworks may be used as well!



# Hive (interface)

- Provides SQL interface to Hadoop (which is coded in Java)
- This simplifies the use of Hadoop for not using Object Oriented Programming (OOP)

Wiki



# HBase (table management)

- Table management system on top of Hadoop
- Allow users to request underlying data like a traditional database (Similar to Big Table in the Google ecosystem)




Wiki

# ZooKeeper (coordinator)

- Provides coordination between the different bricks

Wiki



# Oozie (scheduler)

- Define the scheduling of operations in a workflow, for example in case of start, end, or failure of nodes

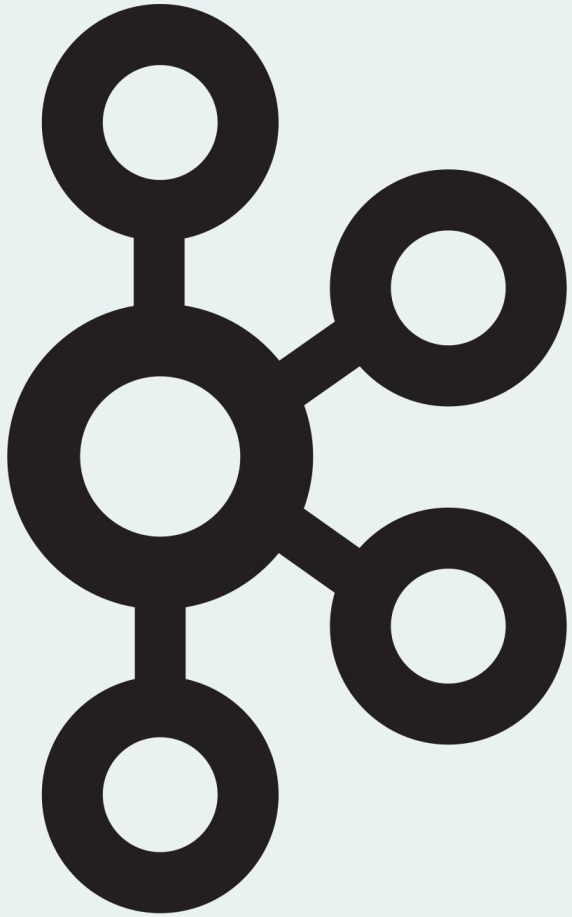
Wiki



# Kafka (event processing)

- Distributed event store and stream-processing platform
- Allow to handle data streaming
- More details about Kafka later in this course

Wiki



# Spark (processing engine)

- Distributed computing engine
- Built-in libraries for most use cases: machine learning, streaming, graph processing...
- More details about Spark later in this course

Wiki



# Flume & Sqoop (data transfer)

- Transfer data between other systems and Hadoop
- Sqoop for relational database
- Flume for unstructured or semi-structured data

Wiki Flume

Wiki Sqoop



# Back to Hadoop

We will focus on :

- The structure of a cluster
- HDFS
- Map Reduce
- YARN

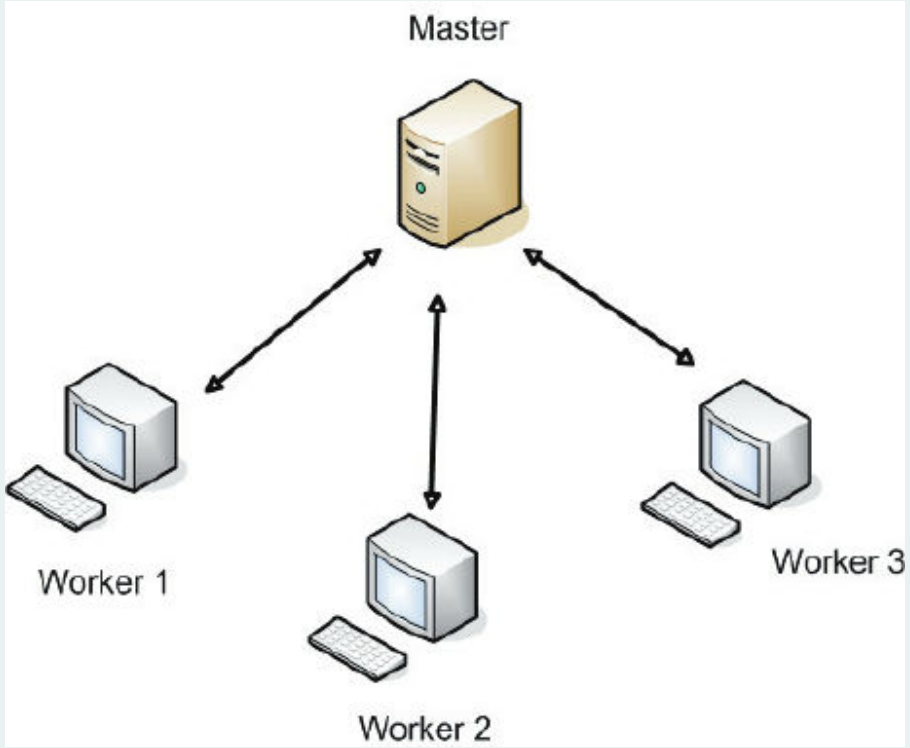
# Hadoop: How does it work?

In a Hadoop cluster, **HDFS (Hadoop Distributed File System)**, **MapReduce**, and **YARN (Yet Another Resource Negotiator)** are three critical components that work together to process and manage large-scale data.

**But first, let's see the different components of an Hadoop cluster**

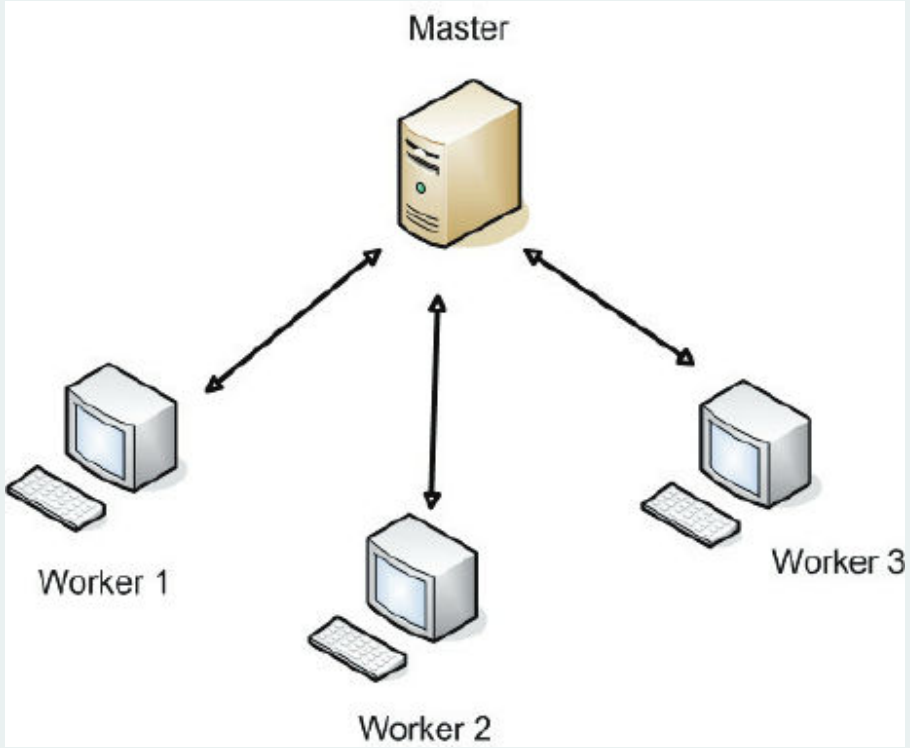
# Definitions: (1/3)

- **Nodes** are individual computing devices within a distributed system or a cluster. Each node can be a physical server, a virtual machine, or a container, depending on the architecture. Nodes can take on different roles, such as master, worker, or storage, depending on their function within the system.



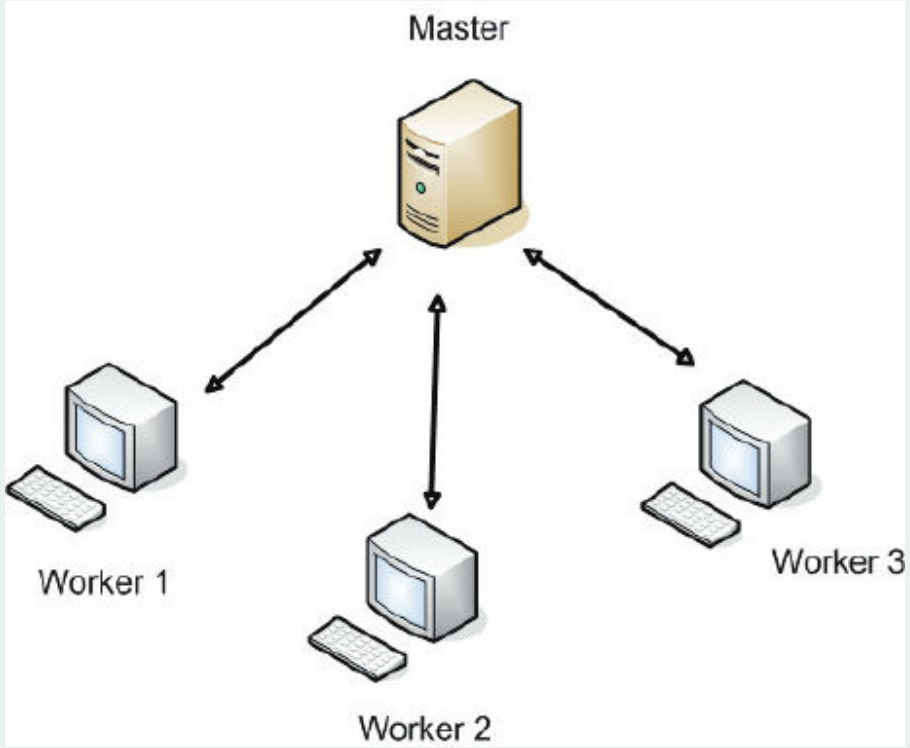
# Definitions: (2/3)

- **Workers**, also known as worker nodes or slave nodes, are the components in a distributed system responsible for performing data processing tasks. They receive instructions, execute the tasks, and return the results.



# Definitions: (3/3)

- **Master**, also known as the master node or coordinator, is a central component responsible for managing, coordinating, and distributing tasks to worker nodes. The master node oversees the overall operation of the system, including task assignment, resource allocation, fault tolerance, and system monitoring.



# Slave nodes in Hadoop

Each node performs a specific task and is responsible for storing and processing data.

There are two types of slave nodes in a Hadoop cluster:

- **DataNode**: Store actual HDFS Data. It manages data blocks and communicates with the NameNode to ensure that data is available and replicated across the cluster. Data nodes work together to provide fault tolerance, ensuring that data is not lost due to hardware failures or other issues.
- **NodeManager**: A NodeManager is responsible for managing resources and executing tasks in Hadoop's processing framework, YARN. It communicates with the ResourceManager to allocate resources and execute tasks based on the application's requirements.

# Master nodes in Hadoop

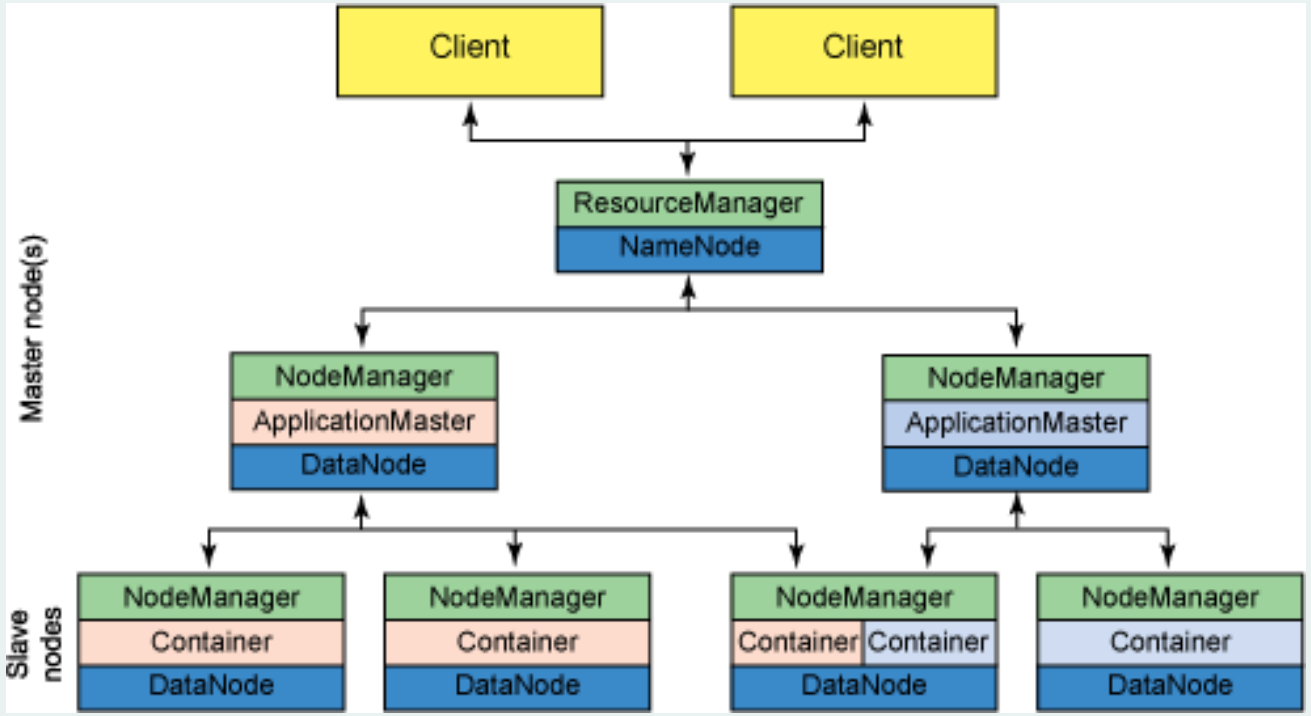
Master nodes are responsible for coordinating and managing the overall functioning of the Hadoop cluster.

There are two types of master nodes:

- **NameNode**: This is the central component of the HDFS, maintaining the metadata for the entire file system, including the location of data blocks and file permissions. It also manages the replication of data blocks across DataNodes to ensure fault tolerance. The NameNode communicates with DataNodes to monitor their health and coordinate data access.
- **ResourceManager**: responsible for managing resources and executing tasks in YARN. It communicates with the NodeManager to allocate resources and execute tasks based on the application's requirements.

# Structure schema

**Note**:
*ApplicationMasters* are specialized instances of NodeManagers used by the ResourceManager to coordinate the execution of tasks for specific applications.



# Overview

Here is a brief overview before we go more in-depth with each concept:

- **HDFS** stores input and output data. When a MapReduce job is submitted, it reads the input data from HDFS, processes it, and writes the output back to HDFS.
- **MapReduce** is a programming model and data processing framework designed for processing and analyzing large datasets in a distributed and parallel manner across a cluster of computers.
- **YARN** is responsible for managing the resources required to run MapReduce jobs. When a MapReduce job is submitted, the ResourceManager in YARN allocates resources.

# Explaining HDFS

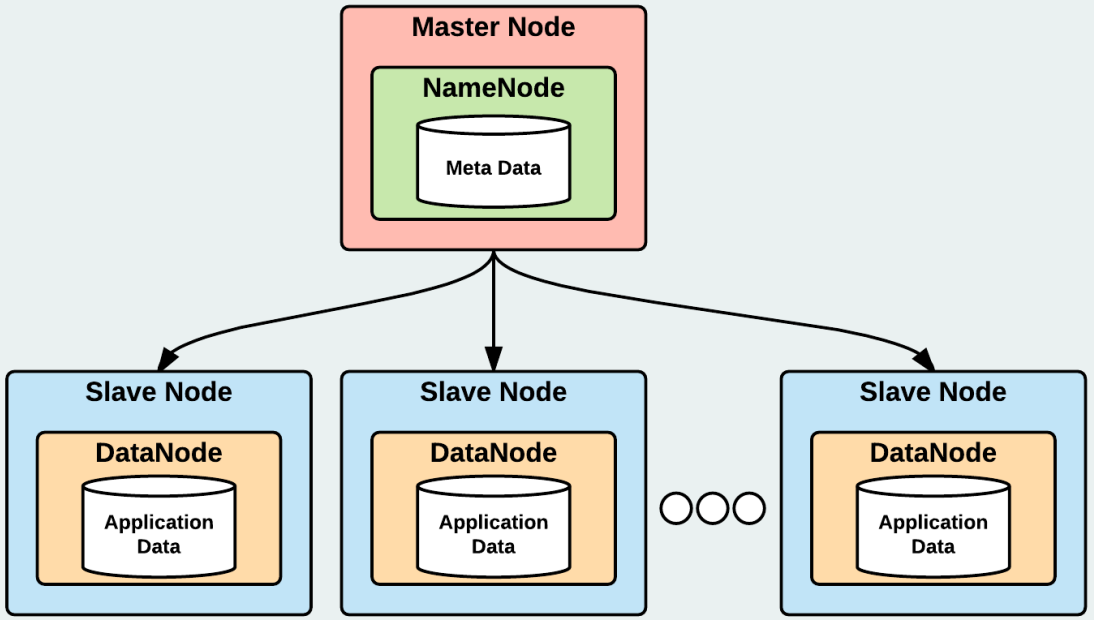
- HDFS, or Hadoop Distributed File System, is a distributed storage system designed to handle large amounts of data across multiple nodes in a cluster.
- It provides high availability, fault tolerance, and scalability by automatically splitting files into blocks, distributing them across the cluster, and replicating them for reliability.

Let's see how it is done!

# Name node & Data nodes

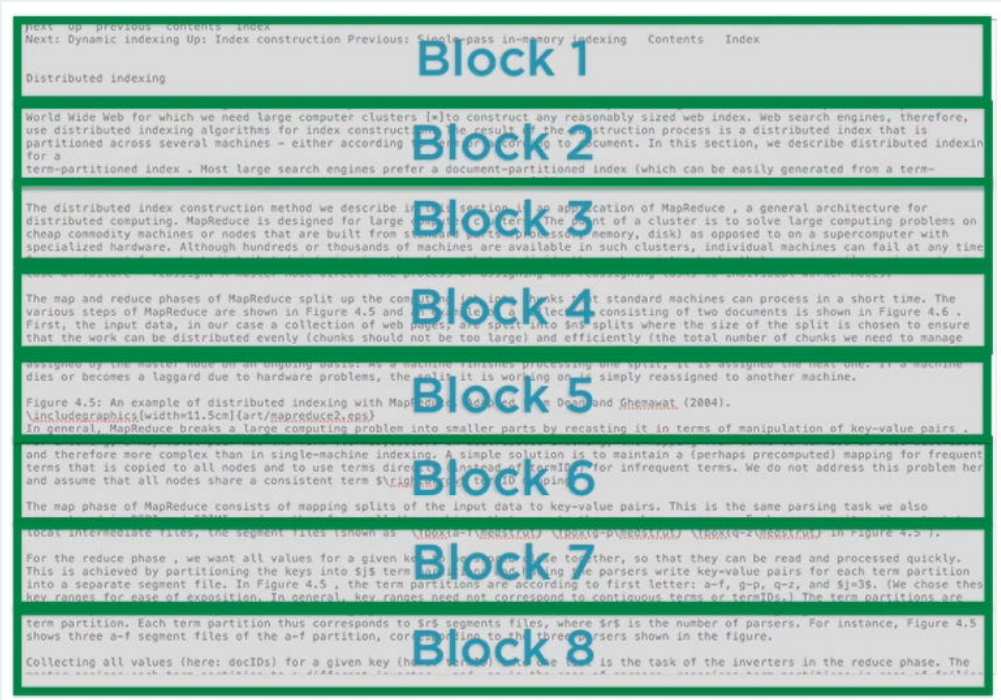
As a reminder, in Hadoop, the data part of the cluster is composed of one Name node (Master), and several Data nodes (Workers or Slaves).

- The Namenode acts as some sort of table of contents, referencing the actual data in the different nodes (directory structure, metadata...).
- It manages the overall file system.



# Storing a file

- Files are often big in size: To facilitate the storage they are broken into smaller data blocks of a given size.
- This simplifies the storage: uniform and standardized unit of storage and the same time to process each block. The seek time is also reduced.
- The default block size in HDFS is 128 MB.



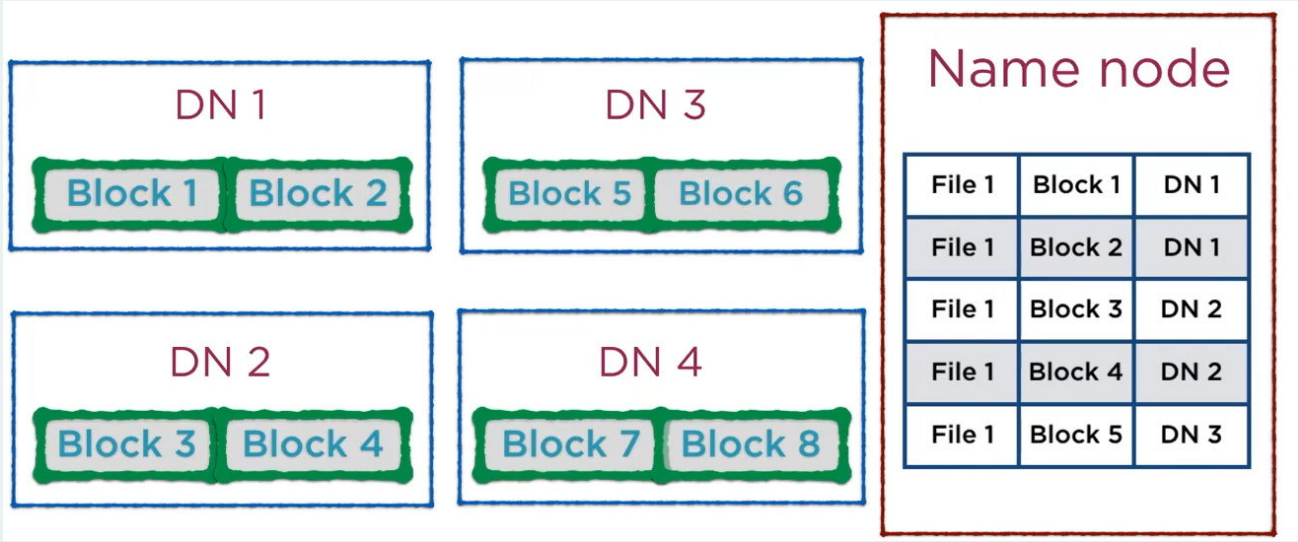
# Block size

The 128 MB size of the blocks is a trade off:

- The smaller the size, the more overhead you have (= a lot of split in the data)
- The bigger the size, the less you are able to parallelize each task (= fewer splits)

# Reading a file

- **Step 1:** Use metadata in the Name node to seek the desired block locations.
- **Step 2:** Read the blocks from each respective location.



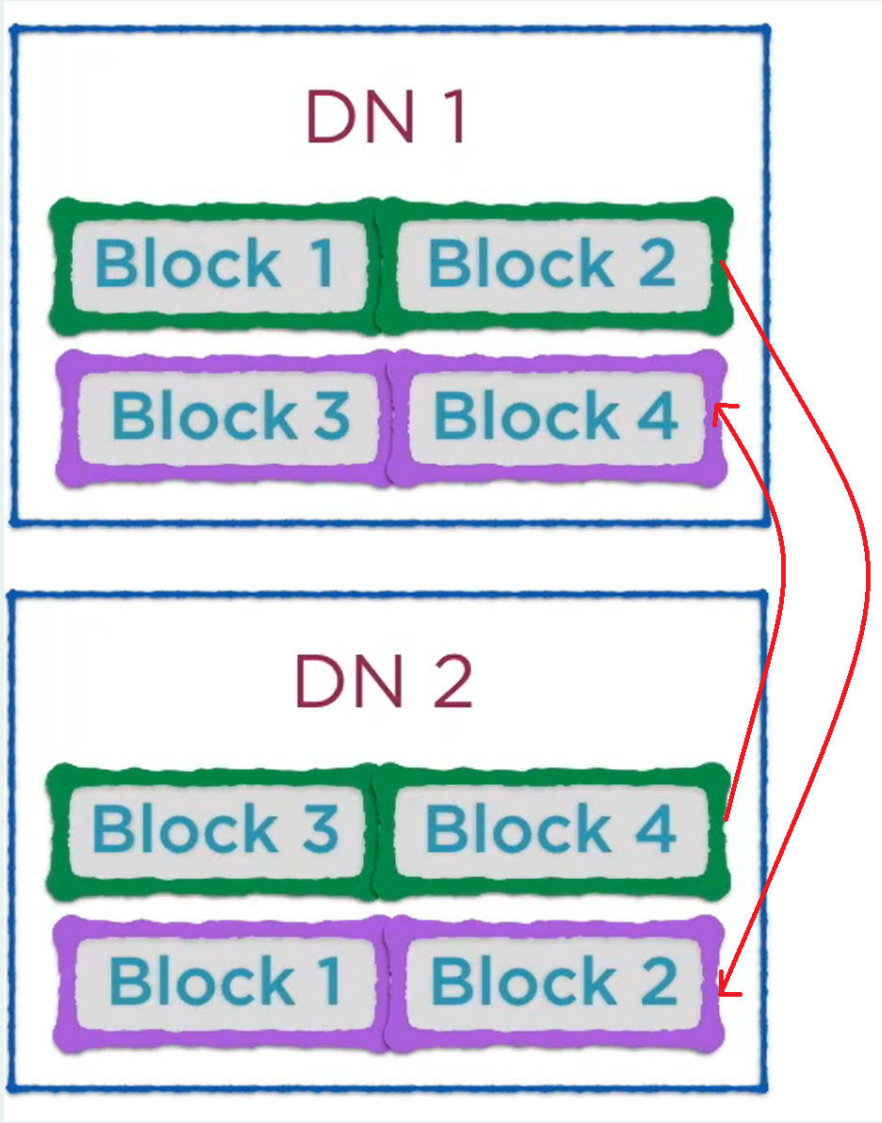
# Fault tolerance: Data nodes

Sometimes, you can lose data for a multitude of reasons, including:

- A Data block may be corrupted
- A data node can be offline or destroyed

For this reason, we define a **replication factor**. Data will be copied multiple times in different locations to help prevent data loss.

Here is an example with a replication factor of 2 ->



# Fault tolerance: Replicas trade-off

When you are choosing your replication options, you are balancing two factors:

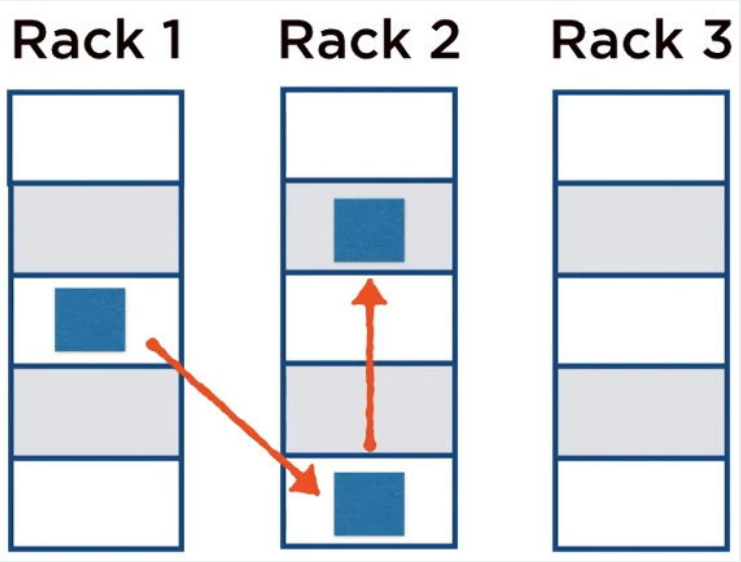
- Maximizing the redundancy to prevent data loss
- Minimizing write bandwidth between racks in a data center

# Fault tolerance: Default location strategy

By default, Hadoop tries to balance those two aspects as follows for a **replication factor of 3** (the default value):

- The **first** location for a block is chosen at random.
- The **second** location will be placed on a different rack if possible
- The **third** location is on the same rack as the second but on a different node.

When a client needs to read a file, he is directed to the closest node.



# Fault tolerance: Name node

The name node is 'the brain' of your cluster. If it goes down, you need to have redundancy or else you could lose all your data.

We have two ways of mitigating this:

- Metadata
- Secondary Name Node

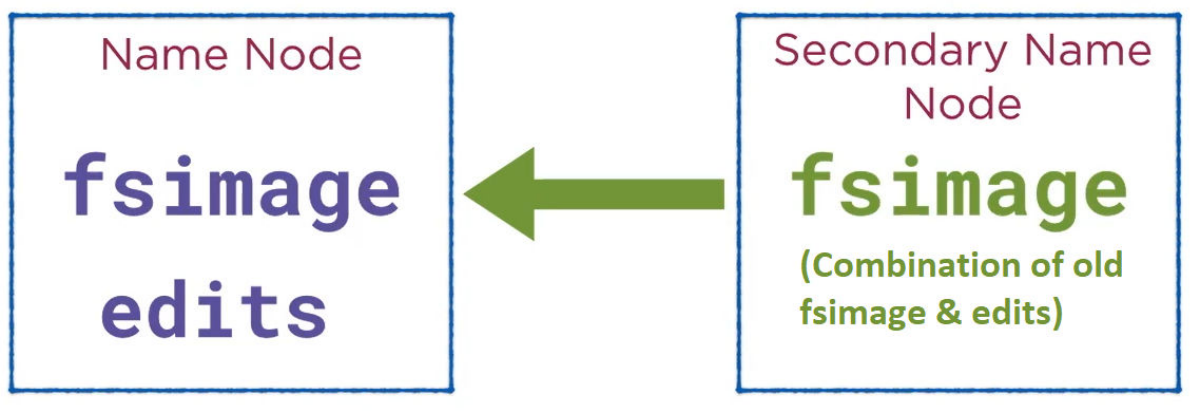
# Metadata recovery

- A snapshot of the file system at startup (named **fsimage**)
- as well as a log file (named **edits**) of all changes to the file system since the last fsimage was made
- Both of them can be stored in a remote location (one or many)
- Merging these two files is reliable, but can take a lot of time, which could leave the system unavailable for a long time



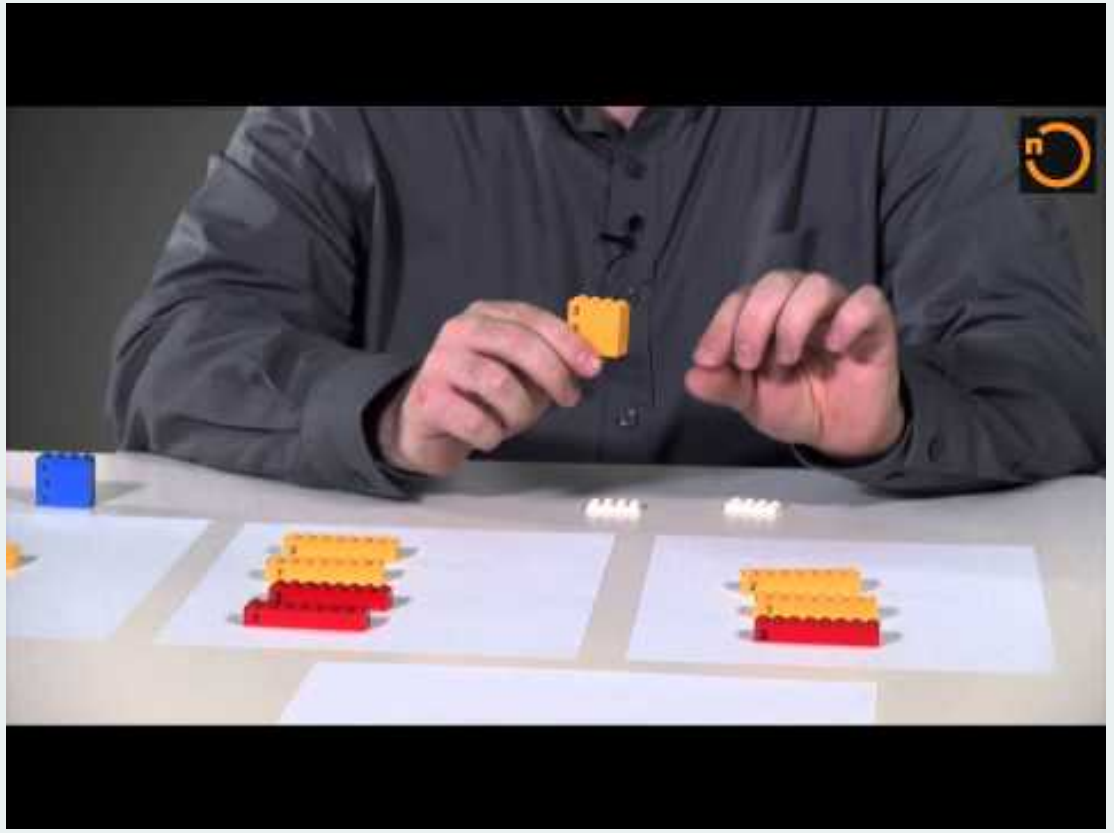
# Secondary Name Node

- This is an exact backup of your primary Name node
- You can sync both systems together by merging the fsimage and edits on the secondary node, then transfer the newly created fsimage (this is called **checkpointing**)
- Your secondary Name node can switch to the primary in case of an emergency



# Understanding HDFS using Legos

Here is a good explanation of HDFS with LEGOs (one demo is worth more than words!)

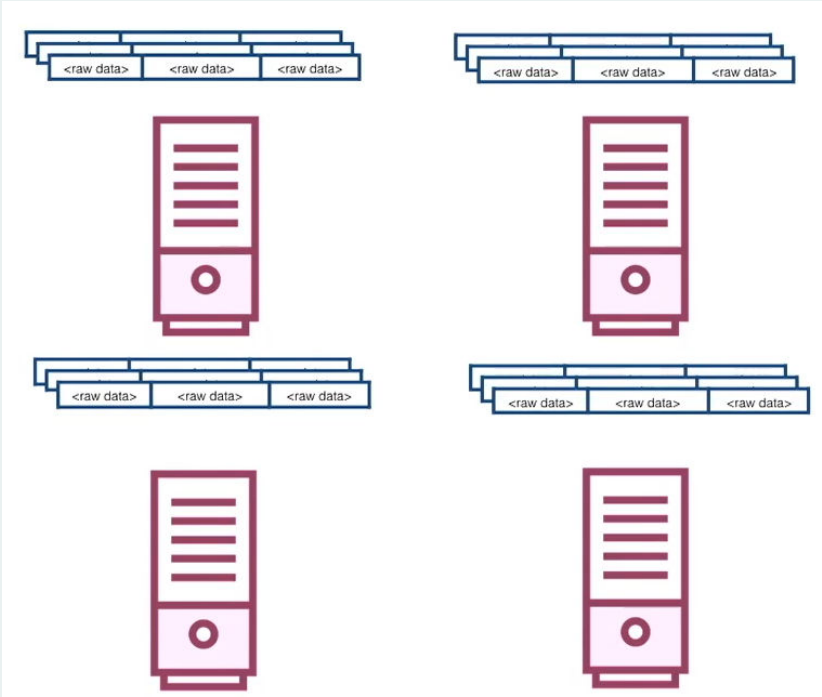


# Explaining MapReduce

- MapReduce is a programming model and data processing technique used for processing and generating large datasets in parallel across a cluster of computers.
- The developer only needs to handle two primary functions: **Map** and **Reduce**. Everything else is managed by Hadoop in the background.

# Map

- The Map function takes an input record (or dataset) and processes each element individually from each node.
- It applies a specific operation or transformation to each input item, generating a set of intermediate key-value pairs.
- The primary purpose of the Map function is to break down the data into smaller chunks that can be processed independently.
- We can run the Map function on each node, as this allows us to not move the data, reducing latency.



# Map example

Imagine your map function counts the number of `A` in a record (non-case-sensitive).

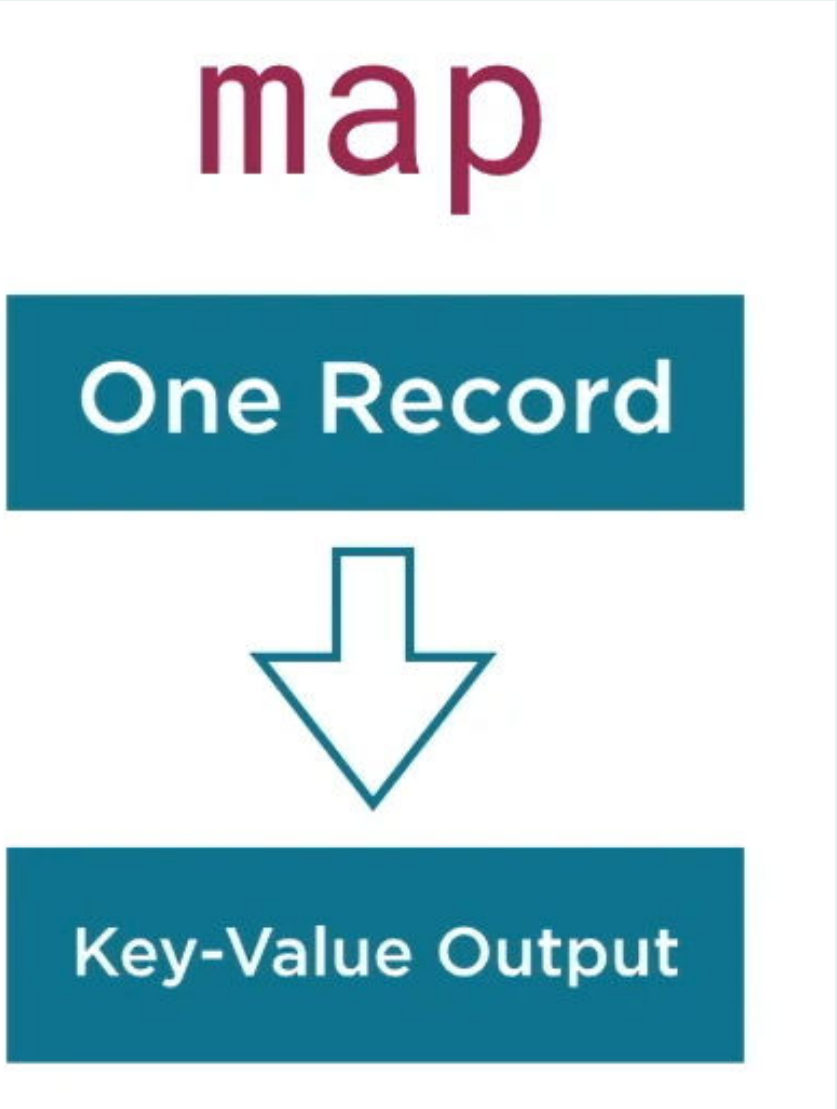
We will take each sentence of the movie `Star Wars: A New Hope` opening crawl as a record.

If you have the first line of text as an input:

> A long time ago, in a galaxy far,
> far, away...

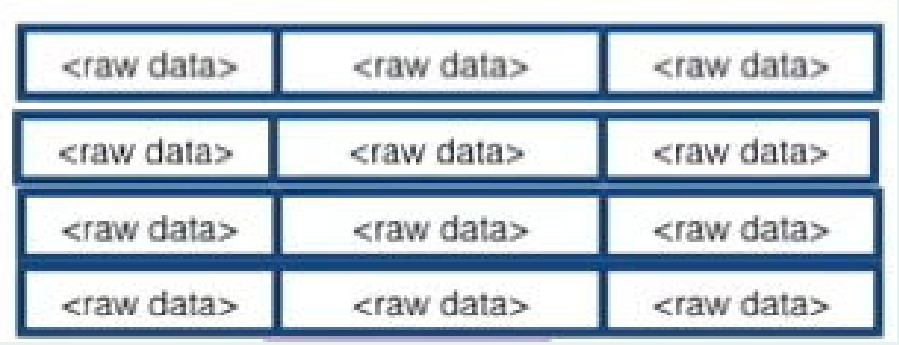
You will have this Key-Value output:

```
{"A": 9}
```



# Reduce

- The Reduce function takes the intermediate key-value pairs generated by the Map function and consolidates them based on their keys.
- It then applies a specific operation or aggregation to the values associated with each key, producing a final output.
- The primary purpose of the Reduce function is to combine the data that was processed independently by the Map function and generate a consolidated result.



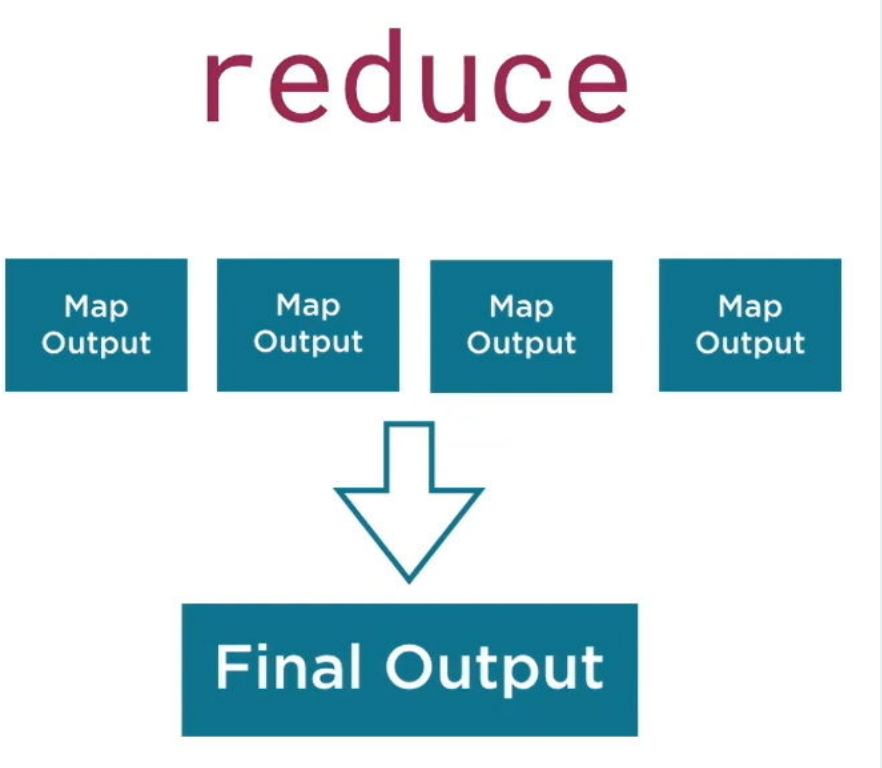
# Reduce example

Let's continue with our `Star Wars: A New Hope` opening crawl.

Each individual map output are as follows:

```
{
  "A": 9,
  "A": 10,
  "A": 14,
  "A": 13
  }
```

The Reduce in this case will sum the different values together, giving a **total output of 46**.



# Shuffle & Sorting

Our reduced example was very simple with only `A` values, but it could have been far more complex, for example counting every occurrence of every letter.

In this case, we had an intermediary step between the Map and Reduce stage: a Shuffle and Sorting.

- It is responsible for redistributing and organizing the intermediate key-value pairs generated by the Map function
- During the shuffle phase, the intermediate key-value pairs are sorted by their keys, and the values for each key are merged.

# Shuffle example

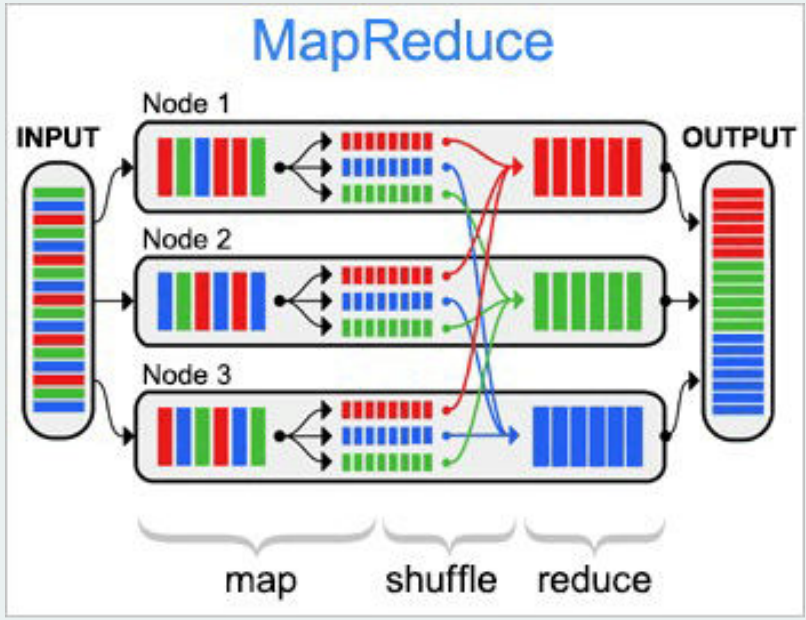
If our Map was counting every letter instead of just `A`, our first input would have been:

> A long time ago, in a galaxy far, far,
> away...

```
{
  "A": 9, "E": 1, "F": 2, "G": 3, "I": 2, "L": 2, "M": 1,
  "N": 2, "O": 2, "R": 2, "T": 1, "W": 1, "Y": 2,
  }
```

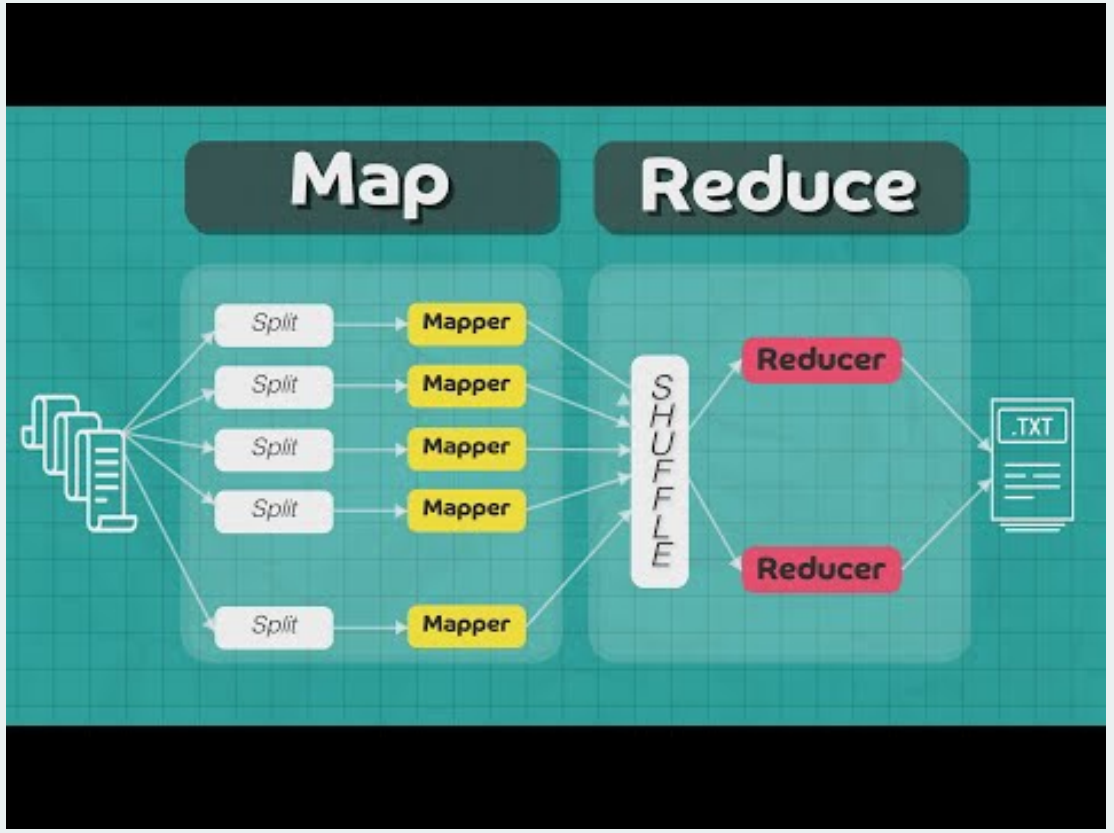
The Shuffle would have created groups of letters `A`, `B`, `C` ... and sent to a different reduce for each letter.

Each reduce would have output the result for one given letter, and the final output is the aggregation of all reduce operations.



# Video presentation of Map reduce

Here is a video explaining MapReduce



# Video presentation of Map reduce

You can now do the `2.2_Lab_Hadoop_docker` that will guide you on how to manipulate Hadoop and do your first map reduce!



# Credit and used resources:

- (Not Free) Pluralsight: The Building Blocks of Hadoop - HDFS, MapReduce, and YARN by Janani Ravi
- Hadoop & Spark courses from previous years by Olivier Moïse (2022) & Fabrice Woittequand (2020)